Form & Function

Fitness AI Coach – Project Report

Christopher Chery
October 29, 2026

Local website: `http://127.0.0.1:5001/`
Public website: Vercel setup prepared; account verification pending.
GitHub repository: `https://github.com/y79mgjphy2-afk/form-and-function`

About the project

Form & Function is a fitness website that puts together a weekly workout plan. The user chooses a goal, experience level, number of training days, and available equipment. The website then shows a plan with exercises, sets, repetitions, and rest times.

The purpose is to give someone a starting point without making them build a routine from scratch. For example, a beginner training at home can select general fitness, three days, and no equipment. Someone with dumbbells can choose that option instead and receive different exercise suggestions.

The project uses Flask, a Python web framework, with HTML, CSS, and JavaScript for the page. It has a demo mode that works without an API key and a live mode that can request a plan from an AI model.

Using the website

The main form offers four goals: general fitness, strength, weight-management support, and mobility. Experience can be beginner, intermediate, or advanced. The form offers two to five training days per week and four equipment choices: no equipment, dumbbells, resistance bands, or a full gym. There is also an optional notes box for preferences or limitations.

After choosing *Build my plan*, the user sees a summary and a card for each workout. A button on each card marks that workout complete, which updates the weekly progress bar. The user can save the plan as a JSON file or print it. Progress is temporary: refreshing the page or generating another plan clears the completion marks.

What the AI does

Demo mode uses exercise templates stored in the Python code. It changes exercises based on the selected goal and equipment. It cycles through three workout templates when more training days are requested. It does not use an AI model, and the optional notes do not change its plans.

Live mode sends the selected profile and notes to the OpenAI Responses API. The model returns a plan as JSON, which the server checks before displaying it. The default model is `gpt-4.1-mini`; this can be changed through a server environment variable.

In this version, the AI makes a plan in one request. It does not choose tools, track the user over time, or revise the plan on its own. It is an AI planning assistant, so more work would be needed if the course requires an autonomous agent with a tool-use loop.

How the application is built

Flask serves the page and handles the plan requests. When the page opens, the JavaScript requests configuration from `/api/config`. This tells the browser whether live mode is available and supplies a token used to check later submissions.

Submitting the form sends the profile to `/api/plan`. The server checks the inputs before doing anything with them. Goals, experience levels, and equipment must match the supported choices. The day count must be an integer between one and five, even though the current form starts at two. Notes are limited to 500 characters.

The server then either creates a demo plan or calls the model provider. For a live response, it checks that the result contains a summary, the requested number of workouts, exercise details, and tips. If the response cannot be read or does not match the expected format, the app returns an error instead of showing an incomplete plan.

Flask renders the page from its templates folder. For Vercel, a build script copies CSS and JavaScript into the public folder for delivery. The JavaScript builds the workout cards from the response. It adds the returned content as text rather than interpreting it as HTML. The completion buttons and progress bar run in the browser; there is no database storing them.

The main project files are:

File	What it contains
`app.py`	Flask routes, input checks, demo plans, and the live AI request.
`templates/index.html`	The profile form and the page structure.
`static/style.css`	Page styling, responsive layout, and print rules.
`static/app.js`	Form submission, plan display, progress, download, and printing.
`tests/test_app.py`	Automated tests for the server.
`requirements.txt`	Python packages needed to run the app.
`vercel.json`	Vercel build settings and function duration.
`prepare_vercel.py`	Copies browser assets to the public folder during deployment.
`README.md`	Instructions for setup and deployment.

API keys and user information

The API key stays on the server in the `OPENAI_API_KEY` environment variable. It should never be included in the page, browser JavaScript, or GitHub repository. The project has a `.gitignore` file to exclude local environment files. The example environment file only describes the settings; the app does not load it automatically.

Live mode sends the profile and optional notes to the model provider, so users should avoid entering private medical information. Plans are not saved to a database. Downloading the JSON file gives the user a copy to keep.

The app also checks a CSRF token on submissions and applies a three-second interval between plan requests in the same session. Before making paid live mode publicly available, it would need login controls and a usage limit across all users.

Testing and current limitations

The project has six automated tests. They check whether demo plans match the requested days and equipment, whether invalid inputs are rejected, and whether requests require a CSRF token. They also check the live path using sample provider responses, including a response with missing fields, and check that the page and static files load with the expected security header.

The tests were run with:

```
python -m unittest discover -s tests -v
```

All six tests passed. The live tests use mocked responses rather than a real API call. That means the response handling was tested, but a real model request and the quality of its exercise suggestions have not been verified for this report.

There are some limits to the current version. Demo plans are based on a small set of templates, and limitation notes do not affect them. Beginner and intermediate demo plans currently use the same set multiplier, while advanced plans increase it. Plans do not adapt based on completed workouts, and completion history disappears when the page is refreshed.

The website gives general fitness suggestions. Its warnings and AI instructions do not guarantee that an exercise is suitable for a person with an injury or health condition. It does not replace advice from a qualified health professional. Checking the structure of an AI response also does not prove that the workout itself is safe or useful.

Running and presenting the project

To run the app, create a Python virtual environment, install the packages in `requirements.txt`, and start `app.py`. The source uses port 5000 by default. Setting the `PORT` variable to 5001 matches the local address used for this report.

The address `http://127.0.0.1:5001/` only works on the computer running the server. The source is available at `https://github.com/y79mgjphy2-afk/form-and-function`. The app has been prepared for Vercel, including its build settings and public assets. Deployment is waiting on hosting-account verification, so no public app URL has been verified yet.

For the class presentation, the demonstration can show a three-day plan with one equipment choice, then a second plan using different equipment. Marking a workout complete shows the progress feature. The JSON download and print buttons show how a user can keep the result. The explanation should make clear which mode is being demonstrated and whether a real AI request is running.

Next steps

The next steps are to complete Vercel account verification, deploy the app, and add the working public link here. A stable SESSION_SECRET must be set on the host so browser sessions work across server instances. Live AI still needs testing with a real provider account. Longer-term improvements could include saved workout history and a follow-up step that lets the user ask for changes to a plan. If an autonomous agent is required for the assignment, the AI workflow also needs to be extended beyond the current single request.